



DBMS: MODULE-3

▼ Introduction to SQL

▼ Overview of the SQL Query language

▼ Data-manipulation language (DML)

The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.

▼ Data Definition Language (DDL)

The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas

- **Integrity.** The SQL DDL includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.
- **View definition.** The SQL DDL includes commands for defining views.

- **Transaction control.** SQL includes commands for specifying the beginning and end points of transactions.
- **Embedded SQL and dynamic SQL.** Embedded and dynamic SQL define how SQL statements can be embedded within general-purpose programming languages, such as C, C++, and Java.
- **Authorization.** The SQL DDL includes commands for specifying access rights to relations and views

▼ SQL Data Definition

The set of relations in a database are specified using a data-definition language (DDL).

The SQL DDL allows specification of not only a set of relations, but also information

about each relation, including:

- The schema for each relation.
- The types of values associated with each attribute.
- The integrity constraints.
- The set of indices to be maintained for each relation.
- The security and authorization information for each relation.
- The physical storage structure of each relation on disk.

▼ Basic Types

The SQL standard supports a variety of built-in types, including:

- `char(n)` : A fixed-length character string with user-specified length n. The full form, `character`, can be used instead.
- `varchar(n)` : A variable-length character string with user-specified maximum length n. The full form, `character varying`, is equivalent.
- `int` : An integer (a finite subset of the integers that is machine dependent). The full form, `integer`, is equivalent.
- `smallint` : A small integer (a machine-dependent subset of the integer type).

- `numeric(p, d)`: A fixed-point number with user-specified precision. The number consists of p digits (plus a sign), and d of the p digits are to the right of the decimal point. Thus, `numeric(3,1)` allows 44.5 to be stored exactly, but neither 444.5 nor 0.32 can be stored exactly in a field of this type.
- `real`, `double precision`: Floating-point and double-precision floating-point numbers with machine-dependent precision.
- `float(n)`: A floating-point number with precision of at least n digits.

▼ Basic Schema Definition

The following command creates a relation department in the database:

```
create table department
(dept name varchar (20),
building varchar (15),
budget numeric (12,2),
primary key (dept name));
```

The above query creates a relation called "department" in the database. It consists of three attributes: "dept name", "building", and "budget", all of which have been specified with their respective data types - varchar (20) for "dept name" and "building"; and numeric (12,2) for "budget". The primary key is the "dept name" attribute, which means that each value in the "dept name" attribute must be unique.

The varchar data type is used for character strings, with the number in parentheses specifying the maximum length of the string. The numeric data type is used for numbers, with the first number specifying the total digits, and the second number specifying the number of digits after the decimal point.

The primary key is used to ensure that each value is unique; this ensures that each tuple in the relation is unique and can be identified individually. With this query, a database of departments can be created, with each department having a unique name, a building, and a budget.

▼ General form of create table

```
create table r
(A1 D1 ,
A2 D 2 ,
...,
An D n ,
<integrity-constraint1 >,
...,
<integrity-constraintk >);
```

The above general form of the create table query is used to create a new relation in the database. It consists of the name of the relation (r), followed by the attributes and their respective data types (A1, A2, ..., An, D1, D2, ..., Dn). The attributes are separated by commas and can be of any valid data type, such as integers, strings, etc. After the attributes and data types, the query may also include one or more integrity constraints. These constraints are used to ensure that the data in the database is valid and consistent; for instance, a primary key constraint ensures that each value in a particular attribute is unique, while a foreign key constraint ensures that the values in two different attributes are related.

- The **primary key** is an integrity constraint that is used to ensure that each value in a particular attribute is unique. It is specified using the `primary key` keyword, followed by the name of the attribute. This ensures that each tuple in the relation is unique and can be identified individually, and that duplicate values in the specified attribute are not allowed.

The **general form of primary key** is as follows:

```
primary key (A1, A2, ..., An)
```

Where A1, A2, ..., An are the attributes of the relation. The primary key constraint ensures that each value in the specified attributes is

unique and that duplicate values are not allowed.

- A **foreign key** is an attribute of one relation (the referencing relation) that refers to the primary key attribute of another relation (the referenced relation). It is specified using the `foreign key` keyword, along with the name of the referenced relation. Foreign keys are used to guarantee the consistency of data across multiple relations in the database. For instance, if an employee tuple references a department tuple via a foreign key, then any updates to the department tuple must also be reflected in the employee tuple.

The general form of the `foreign key` clause is as follows:

```
foreign key (A1, A2, ..., An) references r (P1, P2, ..., Pn)
```

Where A1, A2, ..., An are the attributes of the referencing relation, and P1, P2, ..., Pn are the attributes of the referenced relation (r). The number of attributes in the referencing relation must be the same as the number of attributes in the referenced relation.

- The `NOT NULL` constraint is used to ensure that the specified attribute cannot contain a `NULL` value. This constraint can be used to ensure that the value of the attribute is always known and is not left unspecified. The general form of the `NOT NULL` constraint is as follows:

```
A1 NOT NULL, A2 NOT NULL, ..., An NOT NULL
```

Where A1, A2, ..., An are the attributes of the relation. The `NOT NULL` constraint ensures that the specified attributes do not contain `NULL` values and that all values are known.

▼ ****Primary Key Example****

The following query creates a relation called `employee` in the database, with the `emp_no` attribute specified as the primary key:

```
create table employee
(emp_no int,
name varchar (50),
salary numeric (10,2),
dept_no int,
primary key (emp_no));
```

The query creates a relation named "employee" that consists of four attributes: "emp_no", "name", "salary", and "dept_no". The "emp_no" attribute is specified as the primary key, which means that each value in the "emp_no" attribute must be unique. This ensures that each tuple in the relation is unique and can be identified individually.

▼ ****Foreign Key Example****

The following query creates a relation called `department`, with the `dept_no` attribute specified as a foreign key that references the `emp_no` attribute of the `employee` relation:

```
create table department
(dept_no int,
name varchar (50),
budget numeric (10,2),
manager_emp_no int,
foreign key (dept_no) references employee (emp_no));
```

The query creates a relation named "department" that consists of four attributes: "dept_no", "name", "budget", and "manager_emp_no". The "dept_no" attribute is specified as a foreign key that references the "emp_no" attribute of the "employee" relation. This ensures that the values in the two relations remain consistent with each other, and that any updates to the "emp_no" attribute in the "employee" relation are reflected in the "dept_no" attribute in the "department" relation.

▼ ****Alter Command Example****

The following query adds a new attribute called `location` to the `department` relation:

```
alter table department
add column location varchar (50);
```

The query adds a new attribute called "location" to the "department" relation. The data type of the attribute is specified as "varchar (50)", which means that the attribute can store character strings of up to 50 characters in length.

▼ ****Explaining Table Example****

The following table shows an example of a relation called `employee`, with attributes `emp_no`, `name`, `salary`, and `dept_no`:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

The table above shows an example of a relation called "employee". It consists of four attributes: "emp_no", "name", "salary", and "dept_no". The "emp_no" attribute is the primary key, which means that each value in the "emp_no" attribute must be unique. The "dept_no" attribute is a foreign key that refers to the "dept_no" attribute of another relation. The "name" and "salary" attributes contain the name and salary of the employee, respectively.

▼ **Basic Structure of SQL Queries**

The **SELECT FROM WHERE** statement is a fundamental SQL query that is used to retrieve data from tables in a database. It consists of three clauses: `SELECT`, `FROM`, and `WHERE`.

The **SELECT** clause is used to specify the attributes that should be included in the query results. It consists of a list of attributes, separated by commas.

The **FROM** clause is used to specify the name of the table from which the data should be retrieved.

The **WHERE** clause is used to specify conditions that must be met for the data to be retrieved. It consists of conditions that are combined using logical operators such as **AND**, **OR**, and **NOT**.

▼ Queries on a single relation

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

Instructions: "Write a query to select the name and salary of all employees with a salary greater than 10000"

```
SELECT name,  
FROM employee  
WHERE salary > 10000;
```

Output:

name
Jane Doe

Explain the above query

▼ Queries on Multiple Relation

```
SELECT e.name, e.salary, d.name
FROM employee e, department d
WHERE e.dept_no = d.dept_no
AND e.salary > 10000;
```

Output:

e.name	e.salary	d.name
Jane Doe	12000	Department A

▼ Additional Basic Operations

▼ Rename Operation

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

Instructions: "Write a query to select the name and salary of all employees with a

```
SELECT e.name as Employee_Name, e.salary as Employee_Salary, d.name as Department_Name
FROM employee e, department d
WHERE e.dept_no = d.dept_no
AND e.salary > 10000;
```

Output:

Employee_Name	Employee_Salary	Department_Name
Jane Doe	12000	Department A

The above query uses a **SELECT FROM WHERE** statement to select the name and salary of all employees with a salary greater than 10000. The **SELECT** clause is

used to specify the attributes to be included in the results: "name" from the "employee" relation (aliased as "e"), and "name" from the "department" relation (aliased as "d"). The **FROM** clause is used to specify the tables to be used in the query: "employee" and "department". The **WHERE** clause is used to specify the conditions that must be met for the data to be retrieved: "e.dept_no" must be equal to "d.dept_no", and "e.salary" must be greater than 10000. The **AS** operator is used to rename the attributes in the results, allowing them to be more easily identified.

Distinct Keyword

The **DISTINCT** keyword is used to select unique values from a query result. It is used in the **SELECT** clause of a query, before the list of attributes to be included in the result. For example, if the query is:

```
SELECT DISTINCT e.name, e.salary
FROM employee
```

The query will return a list of distinct employee names and salaries from the "employee" relation. This means that the query will only return unique combinations of employee name and salary; if the same employee name and salary appear more than once, only one instance will be returned. For example, if the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	John Doe	10000	20

The query will return the following result:

name	salary
John Doe	10000
Jane Doe	12000

Since the same "John Doe" and "10000" combination appears twice in the table, only one instance of it will be returned in the query result.

▼ String Operation

Like Keyword

The **LIKE** keyword is used in SQL queries to search for a specific pattern in a string. It is used in the **WHERE** clause of a query, before the string or pattern to be searched for. For example, if the query is:

```
SELECT e.name, e.salary
FROM employee
WHERE e.name LIKE 'John%';
```

The query will return all employee names that begin with the string "John". This means that the query will return only those employee names whose names start with "John"; any names that do not start with "John" will not be returned. For example, if the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	John Smith	8000	20
4	Mary Smith	9000	20

The query will return the following result:

name	salary
John Doe	10000
John Smith	8000

▼ **%** operator

The `%` operation is used in SQL queries to search for a pattern within a string. It is used in the `WHERE` clause of a query, before the string or pattern to be searched for. The `%` operation can be used to search for any character string that matches the pattern before or after the `%` operator.

Example 1

```
SELECT e.name, e.salary
FROM employee
WHERE e.name LIKE '%John%';
```

The query will return all employee names that contain the string "John", regardless of where it appears in the string. This means that the query will return only those employee names whose names contain the string "John"; any names that do not contain "John" will not be returned. For example, if the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	John Smith	8000	20
4	Mary Smith	9000	20

The query will return the following result:

name	salary
John Doe	10000
Jane Doe	12000
John Smith	8000

Example 2

```
SELECT e.name, e.salary
FROM employee
WHERE e.name LIKE '%John';
```

The above query uses a **SELECT FROM WHERE** statement to select all employee names that end with the string "John". , if the table contains the following data:

emp_no	name	salary	dept_no
1	Doe John	10000	10
2	Jane Doe	12000	10
3	Smith John	8000	20
4	Mary Smith	9000	20

The query will return the following result:

name	salary
Doe John	10000
Smith John	8000

Since the name of both employees end with the string "John", they will both be returned in the query result. Any names that do not end with "John" will not be returned.

Example 3

```
SELECT e.name, e.salary
FROM employee
WHERE e.name LIKE 'John%';
```

The query uses a **SELECT FROM WHERE** statement to select all employee names that begin with the string "John". This means that the query will return only those employee names whose names start with "John"; any names that do not start with "John" will not be returned. For example, if the table contains the following data:

--	--	--	--

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	John Smith	8000	20
4	Mary Smith	9000	20

The query will return the following result:

name	salary
John Doe	10000
John Smith	8000

Since the name of both employees begin with the string "John", they will both be returned in the query result. Any names that do not begin with "John" will not be returned.

▼ * operator

The * operator is used in SQL queries to select all attributes from a table. It is used in the **SELECT** clause of a query, before the list of attributes to be included in the result. For example, if the query is:

```
SELECT *
FROM employee;
```

The query will return all attributes from the "employee" relation. This means that all columns and rows of data in the "employee" relation will be included in the query result. For example, if the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20

emp_no	name	salary	dept_no
4	Mary Smith	9000	20

The query will return the following result:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

▼ **ORDER BY** clause

The **ORDER BY** clause is used in SQL queries to sort the results of the query in ascending or descending order. It is used in the **SELECT** clause of a query, after the list of attributes to be included in the result. For example, if the query is:

```
SELECT e.name, e.salary
FROM employee
ORDER BY e.salary;
```

The query will return all employee names and salaries from the "employee" relation, sorted in ascending order by salary. This means that the query will return the employee names and salaries in order from lowest to highest. For example, if the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

The query will return the following result:

name	salary
------	--------

Bob Smith	8000
Mary Smith	9000
John Doe	10000
Jane Doe	12000

▼ **ORDER BY ascending**

To order the results in the ascending order, we need to use the keyword **ASC** in the query. The keyword **ASC** will order the data in an ascending order and will display the results from lowest to highest.

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

```
SELECT e.name, e.salary
FROM employee
ORDER BY e.salary ASC;
```

The above query uses a **SELECT FROM WHERE** statement to select all employee names and salaries from the "employee" relation, sorted in ascending order by salary. This means that the query will return the employee names and salaries in order from lowest to highest.

output

name	salary
Bob Smith	8000
Mary Smith	9000
John Doe	10000
Jane Doe	12000

▼ **ORDER BY descending**

To order the results in the descending order, we need to use the keyword **DESC** in the query. The keyword **DESC** will order the data in a descending order and will display the results from highest to lowest.

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

```
SELECT e.name, e.salary
FROM employee
ORDER BY e.salary DESC;
```

The above query uses a **SELECT FROM WHERE** statement to select all employee names and salaries from the "employee" relation, sorted in descending order by salary. This means that the query will return the employee names and salaries in order from highest to lowest.

output

name	salary
Jane Doe	12000
John Doe	10000
Mary Smith	9000
Bob Smith	8000



▼ WHERE clause predicates

The **WHERE** clause in an SQL query is used to specify the conditions that must be met for the data to be retrieved. These conditions are known as predicates. Predicates are expressions that evaluate to either **TRUE** or **FALSE**. The **WHERE** clause can contain one or more predicates, and the data returned by the query will only include rows where all predicates evaluate to **TRUE**.

There are several types of predicates that can be used in the **WHERE** clause of a query.

▼ Comparison Predicates

Comparison predicates are used to compare the value of two expressions. For example, the **=**, **<**, **>**, **<=**, **>=**, and **<>** operators can be used to compare two values. For example, the following query uses a comparison predicate to select all employees with a salary greater than 10000:

```
SELECT e.name, e.salary
FROM employee
WHERE e.salary > 10000;
```

▼ Logical Predicates

Logical predicates are used to combine multiple predicates in a single **WHERE** clause. The **AND** and **OR** operators can be used to combine predicates. For example, the following query uses a logical predicate to select all employees who are in department 10 and have a salary greater than 10000:

```
SELECT e.name, e.salary
FROM employee
WHERE e.dept_no = 10 AND e.salary > 10000;
```

▼ Pattern Matching Predicates

Pattern matching predicates are used to search for a specific pattern within a string. The **LIKE** keyword can be used to search for a pattern within a string. For example, the following query uses a pattern matching predicate to select all employees whose names begin with the string "John":

```
SELECT e.name, e.salary
FROM employee
WHERE e.name LIKE 'John%';
```

▼ SET operations

SET operations are used in SQL queries to combine the results of multiple queries into a single result set. There are four types of set operations: **UNION**, **INTERSECT**, **EXCEPT** or **MINUS**.

▼ **UNION**

The **UNION** operation is used to combine the results of two or more queries into a single result set. The **UNION** operation eliminates any duplicate rows in the result set. For example, if you have two queries that return the same row, the **UNION** operation will return that row only once. The syntax for the **UNION** operation is:

```
SELECT column1, column2, ...
FROM table1
UNION
SELECT column1, column2, ...
FROM table2;
```

Example

The following query uses the **UNION** operation to combine the results of two queries into a single result set. The two queries retrieve the employee names

and salaries from the "employee" relation, where the employee is in department 10 or department 20.

```
SELECT e.name, e.salary
FROM employee
WHERE e.dept_no = 10
UNION
SELECT e.name, e.salary
FROM employee
WHERE e.dept_no = 20;
```

If the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

The query will return the following result:

name	salary
John Doe	10000
Jane Doe	12000
Bob Smith	8000
Mary Smith	9000

▼ INTERSECT

The **INTERSECT** operation is used to combine the results of two or more queries and return only the rows that appear in all of the queries. The syntax for the **INTERSECT** operation is:

```
SELECT column1, column2, ...
FROM table1
INTERSECT
SELECT column1, column2, ...
```

```
FROM table2;
```

Example

If the table contains the following data:

table employee

emp_no	name	salary	dept_no
1	John Doe	10000	101
2	Jane Doe	12000	102
3	Mary Smith	9000	10

table workers

emp_no	name	salary	dept_no
1	mack	10000	107
2	Jane Dokey	12000	102
3	Bob Smith	8000	20

The following query uses the **INTERSECT** operation to combine the results of two queries into a single result set. The two queries retrieve the employee names and salaries from the "employee" relation, where the employee is in both department 10 and department 20

```
SELECT e.name, e.salary
FROM employee
WHERE e.dept_no = 10
INTERSECT
SELECT e.name, e.salary
FROM workers
WHERE e.dept_no = 20;
```

.

The above query will return the following result:

name	salary

name	salary
Mary Smith	9000
Bob Smith	8000

▼ EXCEPT

The **EXCEPT** operation is used to combine the results of two or more queries and return only the rows that appear in the first query but not in the subsequent queries. The syntax for the **EXCEPT** operation is:

```
SELECT column1, column2, ...
FROM table1
EXCEPT
SELECT column1, column2, ...
FROM table2;
```

Example

The following query uses the **EXCEPT** operation to combine the results of two queries into a single result set. The two queries retrieve the employee names and salaries from the "employee" relation, where the employee is in department 10 but not in department 20.

```
SELECT e.name, e.salary
FROM employee
WHERE e.dept_no = 10
EXCEPT
SELECT e.name, e.salary
FROM employee
WHERE e.dept_no = 20;
```

If the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20

emp_no	name	salary	dept_no
4	Mary Smith	9000	20

The query will return the following result:

name	salary
John Doe	10000
Jane Doe	12000

▼ Aggregate Functions

▼ Average (avg)

Aggregate functions are used in SQL queries to perform calculations on multiple rows of data. The **AVG** function is an aggregate function used to calculate the average value of a set of values. The syntax for the **AVG** function is:

```
SELECT AVG(column_name)
FROM table_name;
```

The **AVG** function returns the average value of the values in the specified column. For example, the following query uses the **AVG** function to calculate the average salary of employees in department 20:

If the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

```
SELECT AVG(employee.salary)
FROM employee
WHERE employee.dept_no = 20;
```

The query will return the average salary of employees in department 20, which is 8500.

▼ Minimum (min)

The `MIN` function is an aggregate function used to calculate the minimum value of a set of values. The syntax for the `MIN` function is:

```
SELECT MIN(column_name)
FROM table_name;
```

The `MIN` function returns the minimum value of the values in the specified column. For example, the following query uses the `MIN` function to calculate the minimum salary of employees in department 20:

If the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

```
SELECT MIN(employee.salary)
FROM employee
WHERE employee.dept_no = 20;
```

The query will return the minimum salary of employees in department 20, which is 8000.

▼ Maximum(max)

The `MAX` function is an aggregate function used to calculate the maximum value of a set of values. The syntax for the `MAX` function is:

```
SELECT MAX(column_name)
FROM table_name;
```

The **MAX** function returns the maximum value of the values in the specified column. For example, the following query uses the **MAX** function to calculate the maximum salary of employees in department 20:

If the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

```
SELECT MAX(employee.salary)
FROM employee
WHERE employee.dept_no = 20;
```

The query will return the maximum salary of employees in department 20, which is 9000.

▼ Total (**SUM**)

The **SUM** function is an aggregate function used to calculate the total sum of a set of values. The syntax for the **SUM** function is:

```
SELECT SUM(column_name)
FROM table_name;
```

The **SUM** function returns the total sum of the values in the specified column. For example, the following query uses the **SUM** function to calculate the total salary of employees in department 20:

If the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

```
SELECT SUM(employee.salary)
FROM employee
WHERE employee.dept_no = 20;
```

The query will return the total salary of employees in department 20, which is 17000.

▼ count(**count**)

The **COUNT** function is an aggregate function used to count the number of rows in a query result set. The syntax for the **COUNT** function is:

```
SELECT COUNT(*)
FROM table_name;
```

The **COUNT** function returns the number of rows in the query result set. For example, the following query uses the **COUNT** function to count the number of employees in department 20:

If the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

```
SELECT COUNT(*)
FROM employee
WHERE employee.dept_no = 20;
```

The query will return the number of employees in department 20, which is 2.

▼ Aggregate with grouping

Aggregate functions can also be used in conjunction with the **GROUP BY** clause. The **GROUP BY** clause is used to group rows of data into separate groups. The **GROUP BY** clause can be used with aggregate functions to calculate an aggregate value for each group of data. For example, the following query uses the **SUM** aggregate function to calculate the total salary of all employees in each department.

If the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

```
SELECT dept_no, SUM(salary)
FROM employee
GROUP BY dept_no;
```

The query will return the total salary for each department, which is:

dept_no	SUM(salary)
10	22000
20	17000

▼ **Having** clause

The **HAVING** clause is used to filter the results of a query based on an aggregate value. The **HAVING** clause is used in conjunction with the **GROUP BY** clause to filter the results of a query based on the aggregate value of a group of rows. For example, the following query uses the **HAVING** clause to filter the results of a query based on the total salary of each department.

If the table contains the following data:

emp_no	name	salary	dept_no
1	John Doe	10000	10
2	Jane Doe	12000	10
3	Bob Smith	8000	20
4	Mary Smith	9000	20

```
SELECT dept_no, SUM(salary)
FROM employee
GROUP BY dept_no
HAVING SUM(salary) > 10000;
```

The query will return the total salary for each department with a total salary greater than 10000, which is:

dept_no	SUM(salary)
10	22000

▼ Aggregation with Null and Boolean Values

Aggregate functions can also be used in queries that contain null or boolean values. When an aggregate function is used with null or boolean values, the result of the aggregate function will be dependent on the type of value.

For example, when using the **SUM** aggregate function with null values, the result will be null. This is because a null value does not have an associated numerical value, so the **SUM** function will not be able to add the null value to the result.

When using the `SUM` aggregate function with boolean values, the result will be the sum of all true values. This is because the boolean value `true` is associated with the numerical value 1, so the `SUM` function will add all true values to the result.

Similarly, when using the `COUNT` aggregate function with null values, the result will be the number of non-null values, as null values are not counted. When using the `COUNT` aggregate function with boolean values, the result will be the number of true values.

▼ Nested Sub-Queries

Nested sub-queries are SQL queries that are embedded within other SQL queries. A nested sub-query is a query that is inside another query and is used to return data that can be used in the outer query. Nested sub-queries can be used to return data from multiple tables or to perform calculations on data from multiple tables. Nested sub-queries can also be used to filter results from the outer query. Nested sub-queries can be used to optimize performance in complex queries by avoiding unnecessary joins and calculations.

▼ Set Membership

Set membership is a concept used in SQL queries to determine if a value is contained in a set of values. The `IN` operator is used to determine set membership in SQL queries. The `IN` operator returns `TRUE` if the value is contained in the set, and `FALSE` if the value is not contained in the set. This operator can be used to filter the results of a query or to compare values in multiple tables. The syntax for the `IN` operator is:

```
value IN ( set )
```

For example, the following query uses the `IN` operator to filter the results of a query to only include employees with a salary of 10000 or 12000:

```
SELECT *  
FROM employee  
WHERE salary IN (10000, 12000);
```

▼ Set Comparison

Set comparison is a concept used in SQL queries to compare two sets of values.

The **SOME** operator is used to compare two sets of values in SQL queries. The **SOME** operator returns **TRUE** if some of the values in the first set are contained in the second set, and **FALSE** if none of the values in the first set are contained in the second set. This operator can be used to filter the results of a query or to compare values in multiple tables. The syntax for the **SOME** operator is:

```
value1 SOME ( value2 )
```

operator	Description	Operates on
=	Equal to.	Any compatible data types
>	Greater than.	Any compatible data types
<	Less than.	Any compatible data types
>=	Greater than equal to.	Any compatible data types
<=	Less than equal to.	Any compatible data types
<>	Not equal to.	Any compatible data types

Some can be used with any of the above operators

Example

name	salary
John Doe	10000
Jane Doe	12000
Bob Smith	8000
Mary Smith	9000

```
SELECT name, salary
FROM employee
WHERE salary > SOME (10000, 12000);
```

The query will return the following result:

name	salary
Jane Doe	12000

▼ Test for Empty Relation

correlated subquery

A correlated subquery is a subquery that uses values from the outer query in its WHERE clause. This type of subquery is used to return data from multiple tables or to perform calculations on data from multiple tables. Correlated subqueries are typically used to filter results from the outer query based on the results of the subquery.

eg:

The **EXISTS** keyword can be used to test whether a relation is empty or not. The **EXISTS** keyword returns **TRUE** if the relation is not empty, and **FALSE** if the relation is empty. The **EXISTS** keyword is usually used in conjunction with a sub-query to test if the sub-query returns any rows.

If the sub-query returns TRUE the main query is evaluated.

The syntax for the **EXISTS** keyword is:

```
SELECT [columns]
FROM table_name
WHERE EXISTS (subquery);
```

For example, the following query uses the **EXISTS** keyword to test if the sub-query returns any rows:

name	salary
------	--------

name	salary
John Doe	10000
Jane Doe	12000
Bob Smith	8000
Mary Smith	9000

```
SELECT name, salary
FROM employee
WHERE EXISTS (SELECT *
              FROM employee
              WHERE salary > 10000);
```

The query will return all employees whose salary is greater than 10000.

▼ Test for the Absence of Duplicate Tuples

The **UNIQUE** keyword can be used to test if a relation contains duplicate tuples or not. The **UNIQUE** keyword returns **TRUE** if the relation does not contain any duplicate tuples, and **FALSE** if the relation contains duplicate tuples. The **UNIQUE** keyword is usually used in conjunction with a sub-query to test if the sub-query returns any duplicate tuples.

The syntax for the **UNIQUE** keyword is:

```
SELECT [columns]
FROM table_name
WHERE UNIQUE (subquery);
```

For example, the following query uses the **UNIQUE** keyword to test if the sub-query returns any duplicate tuples:

name	salary
John Doe	10000

name	salary
Jane Doe	12000
Bob Smith	8000
Mary Smith	9000

```
SELECT name, salary
FROM employee
WHERE UNIQUE (SELECT name
              FROM employee
              WHERE salary > 10000);
```

The query will return all employees whose salary is greater than 10000 without any duplicate tuples.

▼ Sub-queries in the from Clause

Sub-queries can also be used in the **FROM** clause of a query. Sub-queries in the **FROM** clause are used to return a set of rows from one or more tables. Sub-queries in the **FROM** clause can be used to return data from multiple tables or to perform calculations on data from multiple tables. Sub-queries in the **FROM** clause can also be used to filter results from the outer query.

For example, the following query uses a sub-query in the **FROM** clause to filter the results of a query to only include employees with a salary of 10000 or 12000:

name	salary
John Doe	10000
Jane Doe	12000
Bob Smith	8000
Mary Smith	9000

```
SELECT *
FROM (SELECT *
```

```
FROM employee
WHERE salary IN (10000, 12000))
```

The query will return the following result:

name	salary
John Doe	10000
Jane Doe	12000

▼ With Clause

The **WITH** clause is used in SQL queries to create temporary views or tables that can be used in the same query. The **WITH** clause is particularly useful when creating complex queries that require multiple sub-queries or self-joins. The syntax for the **WITH** clause is:

```
WITH temp_view AS (view_query)
```

For example, the following query uses the **WITH** clause to create a temporary view of the **employee** table and then use it in the main query to filter the results of a query to only include employees with a salary of 10000 or 12000:

name	salary
John Doe	10000
Jane Doe	12000
Bob Smith	8000
Mary Smith	9000

```
WITH employee_view AS (SELECT * FROM employee)
SELECT *
FROM employee_view
WHERE salary IN (10000, 12000);
```

The query will return the following result:

name	salary
John Doe	10000
Jane Doe	12000

▼ Scalar Sub queries

Scalar subqueries are SQL queries that return a single value from a single row. Scalar subqueries are typically used to return a single value from a single row of a table, or to perform calculations on data from a single row. Scalar subqueries are used in the **SELECT** clause of a query, and can also be used in the **WHERE** clause to filter results.

The syntax for a scalar subquery is similar to other SQL queries, but the **SELECT** clause must return a single value. For example, the following query uses a scalar subquery to calculate the average salary of all employees:

name	salary
John Doe	10000
Jane Doe	12000
Bob Smith	8000
Mary Smith	9000

```
SELECT AVG(
  (SELECT salary
   FROM employee)
) AS "Average Salary";
```

The query will return the average salary of all employees, which is 9500.

▼ Scalar Without a From Clause

Scalar sub-queries can also be used without a **FROM** clause. When used without a **FROM** clause, the scalar sub-query will return a single value from a single row without performing a join or referencing a table. This type of scalar sub-query can be used to return a single value that is not stored in a table.

For example, the following query uses a scalar sub-query to calculate the sum of two numbers:

```
SELECT (5 + 5) AS "Sum";
```

The query will return the sum of the two numbers, which is 10.

▼ Modification of Database

Modifying a database involves making changes to the structure and/or content of a database. This can include adding, deleting, or modifying tables and columns, as well as inserting, updating, and deleting data. Modifying a database can also involve changing the access privileges of users or creating new users. Modifying a database is a complex task that requires a thorough understanding of the database structure, data types, and access privileges. It is important to be careful when modifying a database, as any mistakes can have serious consequences.

▼ Deletion

Deletion is the process of removing data from a database. This can include deleting entire rows, columns, or tables. It is important to be careful when deleting data from a database, as any mistakes can have serious consequences. Before deleting any data, it is recommended to back up the database in case the deleted data needs to be restored. Deleting data from a database can also be done using the **DELETE** statement in SQL.

A delete command operates on one relation at a time. To delete tuples from multiple relations, one delete command must be used per relation. The where clause can be as complex as a select command's, or left empty.

The syntax for the **DELETE** statement is:

```
DELETE FROM table_name  
WHERE condition;
```

The **WHERE** clause is used to specify the rows that should be deleted. For example, the following query deletes all employees with a salary of 10000:

name	salary
John Doe	10000
Jane Doe	12000
Bob Smith	8000
Mary Smith	9000

```
DELETE FROM employee
WHERE salary = 10000;
```

▼ Insertion

Insertion is a process of adding records into a table in a database. The INSERT statement is used to add new records into a table.

The syntax of the INSERT statement is as follows:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

For example, to insert a new row into the employee table with values for the columns employee_id, first_name, last_name, and salary, the following statement can be used:

```
INSERT INTO employee (employee_id, first_name, last_name, salary)
VALUES (101, 'John', 'Smith', 40000);
```

The above statement will insert a new row into the employee table with the values for the columns employee_id, first_name, last_name, and salary. The row will have the value 101 for the column employee_id, 'John' for the column

first_name, 'Smith' for the column last_name, and 40000 for the column salary.

Also, the INSERT statement can be used to add multiple rows at once. The syntax of the INSERT statement with multiple rows is as follows:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...),
      (value1, value2, value3, ...),
      (value1, value2, value3, ...),
      ...;
```

For example, to insert two rows into the employee table, the following statement can be used:

```
INSERT INTO employee (employee_id, first_name, last_name, salary)
VALUES (101, 'John', 'Smith', 40000),
      (102, 'Jane', 'Doe', 50000);
```

The above statement will insert two rows into the employee table with the values for the columns employee_id, first_name, last_name, and salary. The first row will have the value 101 for the column employee_id, 'John' for the column first_name, 'Smith' for the column last_name, and 40000 for the column salary. The second row will have the value 102 for the column employee_id, 'Jane' for the column first_name, 'Doe' for the column last_name, and 50000 for the column salary.

▼ Updates

Updates are used to modify existing records in a table in a database. The **UPDATE** statement is used to modify existing records in a table.

The syntax for the **UPDATE** statement is:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

For example, the following query updates the salary of employee John Doe to 15000:

name	salary
John Doe	10000
Jane Doe	12000
Bob Smith	8000
Mary Smith	9000

```
UPDATE employee
SET salary = 15000
WHERE name = 'John Doe';
```

The query will update the salary of John Doe to 15000.

▼ Intermediate SQL

▼ Join Expressions

A join expression is a SQL statement that combines two or more tables in a database. Joins are used to retrieve data from multiple tables and can be used to filter the results of a query.

Types of Join Expressions:

▼ Natural Join

A natural join is a type of join expression that combines two tables by matching all columns with the same name. Natural joins are typically used to combine related tables in a database, such as tables that contain customer information and order information. A natural join will return all rows from both tables that have matching column names.

The syntax for a natural join is:

```
SELECT [columns]
FROM table1 NATURAL JOIN table2;
```

For example, the following query uses a natural join to combine two tables, `customer` and `order`, and return all customers who have placed an order:

Table : customer

customer_id	first_name	last_name
1	John	Doe
2	Jane	Doe
3	Bob	Smith

Table : order

order_id	customer_id	order_date
1	1	@March 1, 2020
2	2	@March 2, 2020

```
SELECT first_name, last_name
FROM customer NATURAL JOIN order;
```

The query will return the following result:

first_name	last_name
John	Doe
Jane	Doe

▼ Join conditions

Join conditions are used to specify how two or more tables should be joined. Join conditions are used to determine which rows from one table should be matched with rows from another table. The join condition is specified using the **ON** clause of a join expression.

For example, consider two tables, **customer** and **order**, that have the following columns:

Table : customer

customer_id	first_name	last_name
1	John	Doe
2	Jane	Doe
3	Bob	Smith

Table : order

order_id	customer_id	order_date
1	1	@March 1, 2020
2	2	@March 2, 2020

The following query uses a join condition to return all orders placed by the customer with the id 2:

```
SELECT order_date
FROM customer
INNER JOIN order
ON customer.customer_id = order.customer_id
WHERE customer.customer_id = 2;
```

The above SQL query is using an `INNER JOIN` to combine the `customer` and `order` tables. The `ON` clause of the join specifies the join condition, which is that the `customer_id` column of the `customer` table must match the `customer_id` column of the `order` table. The `WHERE` clause then filters the results of the join to only include rows where the `customer_id` column of the `customer` table is equal to 2. The query will return the `order_date` of any orders placed by the customer with the id 2.

The query will return the following result:

order_date
@March 2, 2020

▼ Outer Join

An outer join is a type of join expression that combines two tables and returns all rows from one table, even if there are no matching rows in the other table. Outer joins are typically used to return all rows from one table, even if there are no matching rows in the other table. Outer joins can be used to combine related tables in a database, such as tables that contain customer information and order information.

Types of Outer Join

There are three types of outer join: left outer join, right outer join, and full outer join.

▼ Left outer join

A left outer join is a type of join expression that combines two tables and returns all rows from the left table (the first table specified in the join expression), even if there are no matching rows in the right table (the second table specified in the join expression). Left outer joins are typically used to return all rows from the left table, even if there are no matching rows in the right table. Left outer joins can be used to combine

related tables in a database, such as tables that contain customer information and order information.

The syntax for a left outer join is:

```
SELECT [columns]
FROM table1 LEFT OUTER JOIN table2
ON condition;
```

For example, the following query uses a left outer join to combine two tables, `customer` and `order`, and return all customers, even if they have not placed an order:

Table : customer

customer_id	first_name	last_name
1	John	Doe
2	Jane	Doe
3	Bob	Smith

Table : order

order_id	customer_id	order_date
1	1	@March 1, 2020
2	2	@March 2, 2020

```
SELECT first_name, last_name
FROM customer LEFT OUTER JOIN order
ON customer.customer_id = order.customer_id;
```

The query will return the following result:

first_name	last_name
John	Doe
Jane	Doe
Bob	Smith

▼ Right outer join

A right outer join is a type of join expression that combines two tables and returns all rows from the right table (the second table specified in the join expression), even if there are no matching rows in the left table (the first table specified in the join expression). Right outer joins are typically used to return all rows from the right table, even if there are no matching rows in the left table. Right outer joins can be used to combine related tables in a database, such as tables that contain customer information and order information.

The syntax for a right outer join is:

```
SELECT [columns]
FROM table1 RIGHT OUTER JOIN table2
ON condition;
```

For example, the following query uses a right outer join to combine two tables, `customer` and `order`, and return all orders, even if they have not been placed by a customer:

Table : customer

customer_id	first_name	last_name
1	John	Doe
2	Jane	Doe
3	Bob	Smith

Table : order

order_id	customer_id	order_date
1	1	@March 1, 2020
2	2	@March 2, 2020

```
SELECT order_date
FROM customer RIGHT OUTER JOIN order
ON customer.customer_id = order.customer_id;
```

The query will return the following result:

order_date
@March 1, 2020
@March 2, 2020

▼ Full outer join

A full outer join is a type of join expression that combines two tables and returns all rows from both tables, even if there are no matching rows in the other table. Full outer joins are typically used to return all rows from both tables, even if there are no matching rows in the other table. Full outer joins can be used to combine related tables in a database, such as tables that contain customer information and order information.

The syntax for a full outer join is:

```
SELECT [columns]
FROM table1 FULL OUTER JOIN table2
ON condition;
```

For example, the following query uses a full outer join to combine two tables, `customer` and `order`, and return all customers and all orders, even if there are no matching rows in the other table:

Table : customer

customer_id	first_name	last_name
1	John	Doe
2	Jane	Doe
3	Bob	Smith

Table : order

order_id	customer_id	order_date
1	1	@March 1, 2020

2	2	@March 2, 2020
---	---	----------------

```
SELECT first_name, last_name, order_date
FROM customer FULL OUTER JOIN order
ON customer.customer_id = order.customer_id;
```

The query will return the following result:

first_name	last_name	order_date
John	Doe	@March 1, 2020
Jane	Doe	@March 2, 2020
Bob	Smith	

▼ Views

▼ View Definition

A view is a virtual table in a database that is created by a SQL query. Views are used to retrieve data from multiple tables and can be used to simplify complex queries. Views can also be used to limit access to certain data by restricting the columns and rows that are returned. Views are typically used to simplify the process of retrieving data from a database.

The syntax for creating a view is:

```
CREATE VIEW view_name AS
SELECT [columns]
FROM table_name
WHERE condition;
```

For example, the following query creates a view named `customer_orders` that returns the first name, last name, and order date of all customers who have

placed an order:

Table : customer

customer_id	first_name	last_name
1	John	Doe
2	Jane	Doe
3	Bob	Smith

Table : order

order_id	customer_id	order_date
1	1	@March 1, 2020
2	2	@March 2, 2020

```
CREATE VIEW customer_orders AS
SELECT first_name, last_name, order_date
FROM customer INNER JOIN order
ON customer.customer_id = order.customer_id;
```

The query will create a view named `customer_orders` that returns the first name, last name, and order date of all customers who have placed an order.

first_name	last_name	order_date
John	Doe	@March 1, 2020
Jane	Doe	@March 2, 2020

The view can then be used to simplify the retrieval of customer order data from the database.

▼ Materialized view

Materialized views are pre-computed views in databases that store the results of a query. They are used to improve the performance of data access, such as when data is accessed frequently. Materialized views can be used for various purposes, such as providing access to data faster, or providing access to data that may not be available in its raw form. The data stored in a

materialized view is stored in a physical table. The view is then defined with a query that is used to retrieve the data from the table. The query is stored in the database and can be used to refresh the view when necessary.

The process of keeping the materialized view up-to-date is called materialized view maintenance (or often, just view maintenance).

▼ Update of a view

▼ Integrity Constraints

- Integrity constraints are a set of rules. It is used to maintain the quality of information.
- Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.
- Thus, integrity constraint is used to guard against accidental damage to the database.

▼ Integrity Rules

- Data must be stored in the correct format.
- Data must be consistent with each other.
- Data must be valid and accurate.
- Data must be unique.
- Data must be stored according to certain rules and constraints.
- Data must be protected from unauthorized access.
- Data must be protected from accidental or intentional modification.

There are four main types of integrity constraints: domain constraints, entity integrity constraints, referential integrity constraints, and key constraints

▼ 1. Domain constraints

- Domain constraints can be defined as the definition of a valid set of values for an attribute.
- The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

▼ 2. Entity integrity constraints

- The entity integrity constraint states that primary key value can't be null.
- This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- A table can contain a null value other than the primary key field.

Example:

EMPLOYEE

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

▼ 3. Referential Integrity Constraints

- A referential integrity constraint is specified between two tables.
- In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

Example:

(Table 1)

EMP_NAME	NAME	AGE	D_No
1	Jack	20	11
2	Harry	40	24
3	John	27	18
4	Devil	38	13

Foreign key

Not allowed as D_No 18 is not defined as a Primary key of table 2 and In table 1, D_No is a foreign key defined

Relationships

(Table 2)

Primary Key

<u>D_No</u>	D_Location
11	Mumbai
24	Delhi
13	Noida

▼ 4. Key constraints

- Keys are the entity set that is used to identify an entity within its entity set uniquely.
- An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and null value in the relational table.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1002	Morgan	8 th	22

Not allowed. Because all row must be unique

▼ Authorization

Authorization is the process of granting a user access to data and other system resources. It is a security measure that helps to ensure that only authorized users have access to sensitive information. Authorization is typically done through authentication, which is the process of verifying a user's identity. Once a user is authenticated, they are assigned a certain level of access to the system. This means that the user will be able to access only certain areas or features of the system that they are authorized to use. Authorization is an important part of any database management system, as it helps to ensure that data is only accessed by authorized users.

Authorization is required to perform certain operations on the data stored in the database. These operations include the following:

- Authorization to read data.
- Authorization to insert new data.
- Authorization to update data.
- Authorization to delete data.

Each of these types of authorizations is called a **privilege**.

▼ Granting and revoking privileges

Granting

In order to grant and revoke privileges, a user must have the necessary permissions. These permissions can be granted and revoked using the SQL `GRANT` and `REVOKE` statements.

The `GRANT` statement is used to give a user access to certain privileges. The syntax for granting privileges is:

```
GRANT [privileges] ON [table] TO [user];
```

The `privileges` parameter specifies the type of access the user has, such as read, write, or delete. The `table` parameter specifies the name of the table the user has access to, and the `user` parameter specifies the name of the user the privileges are being granted to.

Revoking

The `REVOKE` statement is used to revoke privileges that have been previously granted to a user. The syntax for revoking privileges is:

```
REVOKE [privileges] ON [table] FROM [user];
```

The `privileges` parameter specifies the type of access to be revoked, the `table` parameter specifies the name of the table the privileges are being revoked from, and the `user` parameter specifies the name of the user the privileges are being revoked from.

▼ Roles

A role is a collection of privileges that can be granted to a user or group of users. A role allows users to access resources in a database in a consistent way. For example, a user with the administrator role may have access to all resources in a database, while a user with the reader role may only have access to certain resources.

For example, the `admin` role may have access to all tables in a database, while the `reader` role may only have access to certain tables. In this example, the `admin` role would have `SELECT`, `INSERT`, `UPDATE`, and `DELETE` privileges on

all tables in the database, while the `reader` role would only have `SELECT` privileges on certain tables.

▼ Authorization on Views

Views are used to restrict access to certain data, as well as to ensure data integrity. Authorization on views can be used to limit the access to certain users.

The following is an example of a SQL query to grant authorization on a view:

```
GRANT SELECT ON view_name  
TO username;
```

This query grants the `SELECT` permission on a view to a certain user. This will allow the user to read the contents of the view.

▼ Authorization on Schema

Authorization on a database schema is the process of allowing certain users or groups of users to access certain objects within the schema. This is done by granting the appropriate privileges to the user or group of users.

The privilege to access database objects within a schema is typically granted using the `GRANT` statement. The following is an example of a SQL query to grant authorization on a schema:

```
GRANT [privileges] ON [schema_name] TO [username];
```

The `privileges` parameter specifies the type of access the user has, such as read, write, or delete. The `schema_name` parameter specifies the name of the schema the user has access to, and the `username` parameter specifies the name of the user the privileges are being granted to.

For example, the following statement grants the `SELECT` privilege to the `john` user on the `sales` schema:

```
GRANT SELECT ON sales TO john;
```

This statement will allow the `john` user to access the objects in the `sales` schema and perform `SELECT` operations on them.

▼ Transfer of privileges

Transfer of privileges is the process of granting access to certain resources or privileges to another user or group of users. The transfer of privileges is typically done using the `GRANT` and `REVOKE` statements in SQL.

The `GRANT` statement is used to give a user access to certain privileges. The syntax for granting privileges is:

```
GRANT [privileges] ON [table] TO [user];
```

The `REVOKE` statement is used to revoke privileges that have been previously granted to a user. The syntax for revoking privileges is:

```
REVOKE [privileges] ON [table] FROM [user];
```

By using the `GRANT` and `REVOKE` statements, privileges can be transferred from one user or group of users to another. This allows for more secure access to resources, as only the users with the appropriate privileges will be able to access them.

▼ Revoking of privileges

- Revoking of privileges is the process of removing access to certain resources or privileges from a user or group of users. The revoking of privileges is typically done using the `REVOKE` statement in SQL.

The **REVOKE** statement is used to revoke privileges that have been previously granted to a user. The syntax for revoking privileges is:

```
REVOKE [privileges] ON [table] FROM [user];
```

By using the **REVOKE** statement, privileges can be revoked from a user or group of users. This helps to ensure that only authorized users have access to sensitive information.

- The **REVOKE** statement can also be used with the **RESTRICT** keyword, which prevents the revocation of privileges if any dependent objects exist. The syntax for revoking privileges with the **RESTRICT** keyword is:

```
REVOKE [privileges] ON [table] FROM [user] RESTRICT;
```

By using the **RESTRICT** keyword, the **REVOKE** statement will only succeed if there are no dependent objects that rely on the privileges being revoked. This ensures that the data integrity and security of the system is maintained.

- The **REVOKE** statement can also be used with the **GRANT** keyword, which restricts the revocation of privileges to only the **GRANT** statement. The syntax for revoking privileges with the **GRANT** keyword is:

```
REVOKE [privileges] ON [table] FROM [user] GRANT;
```

By using the `GRANT` keyword, the `REVOKE` statement will only succeed if the privileges being revoked were granted using the `GRANT` statement. This ensures that the privileges that were granted are the only ones that will be revoked.

▼ Row-Level Authorization

Row-level authorization is a security feature of a database management system that allows users to access only the data that they are authorized to view. This means that users will have access to only the specific rows of data that they are authorized to view. This feature is useful for protecting sensitive data, as it ensures that users can only view the data that they are authorized to view.

Row-level authorization is typically implemented using SQL queries. For example, the following query can be used to grant authorization to a user to view only the rows they are authorized to view:

```
GRANT SELECT ON table_name
TO username
WHERE condition;
```

This query grants the user access to only the rows that match the condition in the `WHERE` clause.

Row-level authorization can also be implemented in other ways. For example, some database management systems may support the use of views to restrict access to certain rows. Views can be used to create a virtual table that only displays the rows that the user is authorized to view. This allows the user to access only the data they are authorized to view without having to use a `WHERE` clause in the query.